MCP as a Process Boundary

Author: Konrad Kowalczyk
Repository: ai-agents-JAVA-SPRING

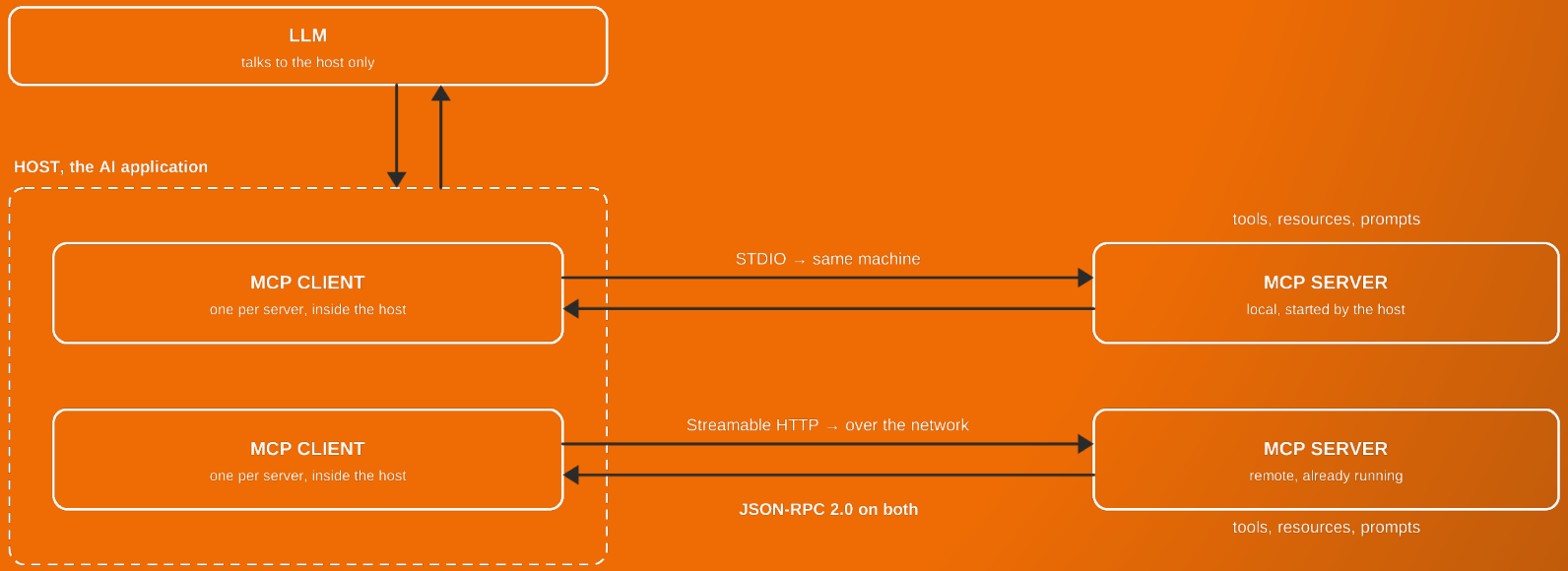
Agenda

1. What is MCP?
2. Claude Desktop setup
3. STDIO
4. What MCP gives you
5. MCP over HTTP
6. Where it breaks



What really is MCP?

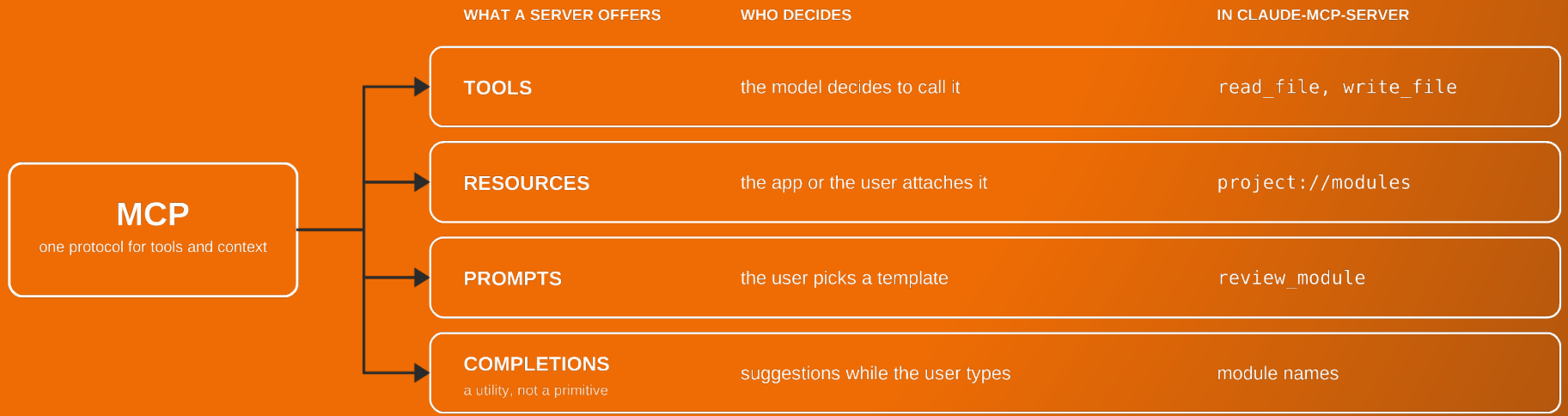
An open protocol that lets an AI application use tools and data that live in another process.



What a server can offer

The server decides what to offer.

The client decides what to use.



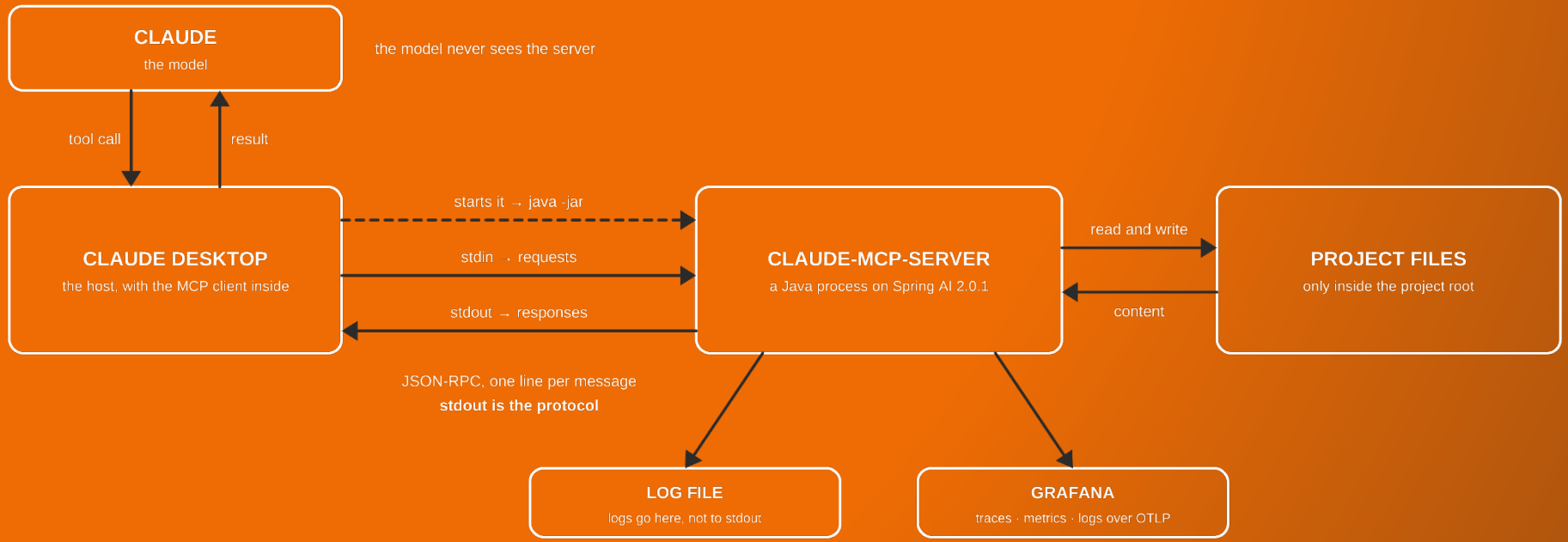
The server speaks MCP 2025-11-25

Client and server agree on the protocol version at startup, and Claude Desktop and this server settle on 2025-11-25.

Spring AI 2.0.1 and **LangChain4j 1.16.3** speak the same version, so every MCP client and server in this repo agrees on it.

Claude + Java ♡

A copilot for your own repo, built from one Spring Boot app.

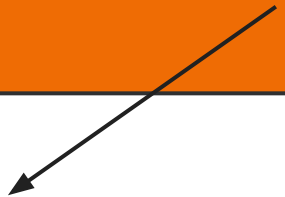


Claude Desktop setup

Add this entry to **`claude_desktop_config.json`** and restart Claude Desktop.

It starts the server itself.

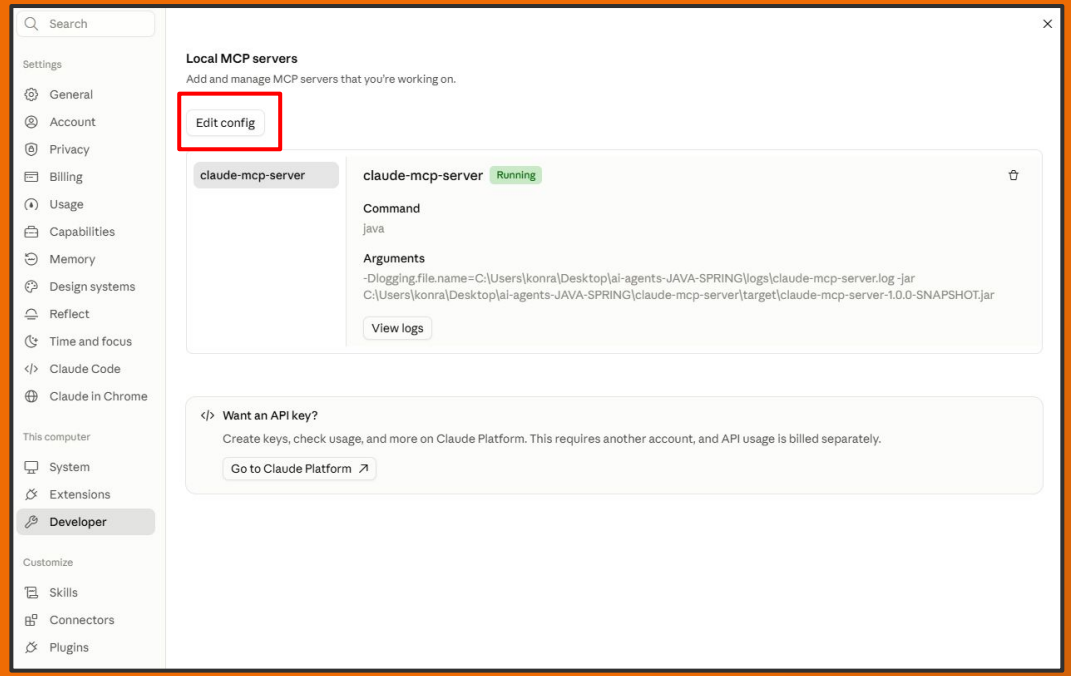
Your Jar Location



```
{
  "mcpServers": {
    "claude-mcp-server": {
      "command": "java",
      "args": [
        "-Dlogging.file.name=C:\\Users\\konra\\Desktop\\ai-agents-JAVA-SPRING\\logs\\claude-mcp-server.log",
        "-jar",
        "C:\\Users\\konra\\Desktop\\ai-agents-JAVA-SPRING\\claude-mcp-server\\target\\claude-mcp-server-1.0.0-SNAPSHOT.jar"
      ]
    }
  }
}
```

Where the config lives

Settings → Developer → Edit Config → add the entry → restart Claude Desktop



Spring Boot configuration

Three keep stdout silent

One line turns STDIO on

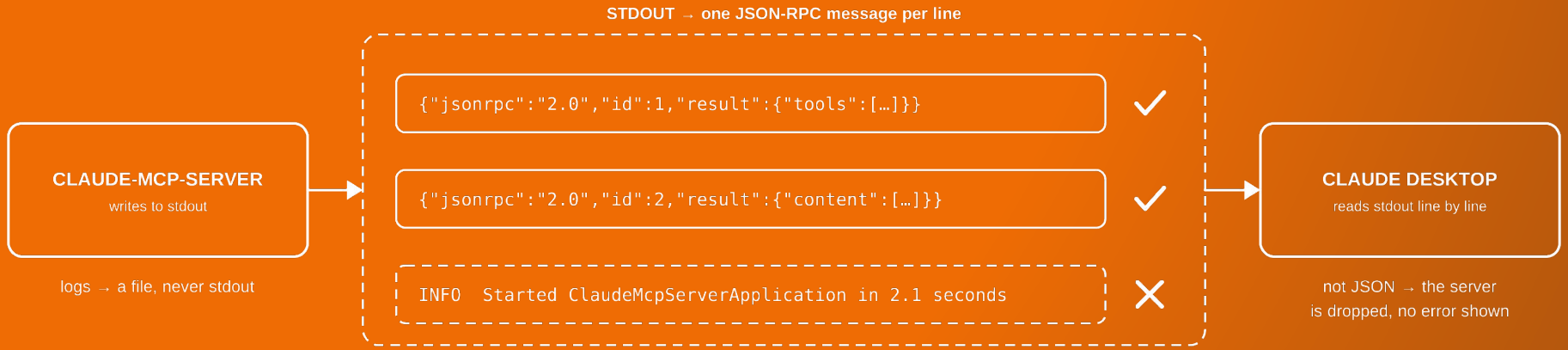
```yaml
spring:
  main:
    banner-mode: "off"              # stdout is the protocol, no banner
    web-application-type: none      # no Tomcat writing to stdout
  ai:
    mcp:
      server:
        name: ai-agents-claude-server
        stdio: true                 # the STDIO switch, there is no protocol: STDIO
        type: SYNC
        capabilities:               # what the server advertises
          tool: true
          resource: true
          prompt: true
          completion: true
        annotation-scanner:
          enabled: true             # finds @McpTool, @McpResource, @McpPrompt, @McpComplete

logging:
  console:
    enabled: false                  # nothing may reach stdout
  file:
    name: logs/claude-mcp-server.log
```

Stdout is the protocol

Every line on stdout must be a JSON-RPC message.

One log line, and Claude Desktop drops the server.



JSON-RPC

One line, one message.

A notification has no id and gets no answer.

- id → the answer comes back with the same id
- method → just a name, not code, the SDK maps it to a handler
- params.name → your @McpTool method

```
→ stdin
{"jsonrpc":"2.0","id":7,"method":"tools/call","params":{"name":"read_file","arguments":{"path":"pom.xml"}}}

← stdout
{"jsonrpc":"2.0","id":7,"result":{"content":[{"type":"text","text":"<project …"}]}}
```

Two locks on one door

`logging.console.enabled` silences only Spring Boot's default setup.
A custom `logback-spring.xml` replaces it, so this file simply has no console appender.

```
<configuration>
    <!-- no CONSOLE appender: stdout carries the JSON-RPC stream -->
    <include resource="org/springframework/boot/logging/logback/defaults.xml"/>
    <include resource="org/springframework/boot/logging/logback/file-appender.xml"/>

    <appender name="OTEL"
              class="io.opentelemetry.instrumentation.logback.appender.v1_0.OpenTelemetryAppender"/>

    <root level="INFO">
        <appender-ref ref="FILE"/>      <!-- logs/claude-mcp-server.log -->
        <appender-ref ref="OTEL"/>      <!-- Loki, over OTLP -->
    </root>
</configuration>
```

What the server announces at startup

Every capability is switched on in one line and registered in the next.

```
18:50:52.552  Starting ClaudeMcpServerApplication using Java 21 with PID 13140 (… started by konra in C:\WINDOWS\system32)
18:50:57.174  Enable tools capabilities, notification: true
18:50:57.631  Registered tools: 9
18:50:57.631  Enable resources capabilities, notification: true
18:50:57.642  Registered resources: 2
18:50:57.642  Enable resources templates capabilities, notification: true
18:50:57.649  Registered resource templates: 2
18:50:57.649  Enable prompts capabilities, notification: true
18:50:57.663  Registered prompts: 4
18:50:57.663  Enable completions capabilities
18:50:57.674  Registered completions: 3
18:50:58.486  Started ClaudeMcpServerApplication in 8.55 seconds (process running for 11.306)
```

The handshake, as Claude Desktop saw it

The client waits for the JVM.

After that, three lists in a quarter of a second.

```
10:52:39.647  Message from client: method="initialize" id=0
10:52:51.122  Message from server: id=0 result                    // 11.5 s, Spring Boot is starting
10:52:51.132  Message from client: method="notifications/initialized"
10:52:51.195  Message from client: method="tools/list" id=1
10:52:51.218  Message from client: method="prompts/list" id=2
10:52:51.219  Message from client: method="resources/list" id=3
10:52:51.418  Message from server: id=1 result
10:52:51.441  Message from server: id=2 result
10:52:51.452  Message from server: id=3 result                    // ready, 250 ms after the handshake
```

What MCP gives you



Tools

@McpTool becomes the contract.

The name, the description and the parameters are all the model will ever know about this method.

```
@McpTool(name = "create_file",                                  // the name the model sees
        description = "Create a new file. Fails if it already exists — "
        + "use write_file to overwrite. Restricted to allowed extensions.")
public String create_file(
        @McpToolParam(description = "Relative path for the new file", required = true)
        String path,                                            // becomes the JSON schema
        @McpToolParam(description = "Initial file content", required = true)
        String content) {
    return traced("create_file", path, len(path) + len(content),
            () -> fileService.createFile(path, content));
}
```

Nine tools, three levels of risk

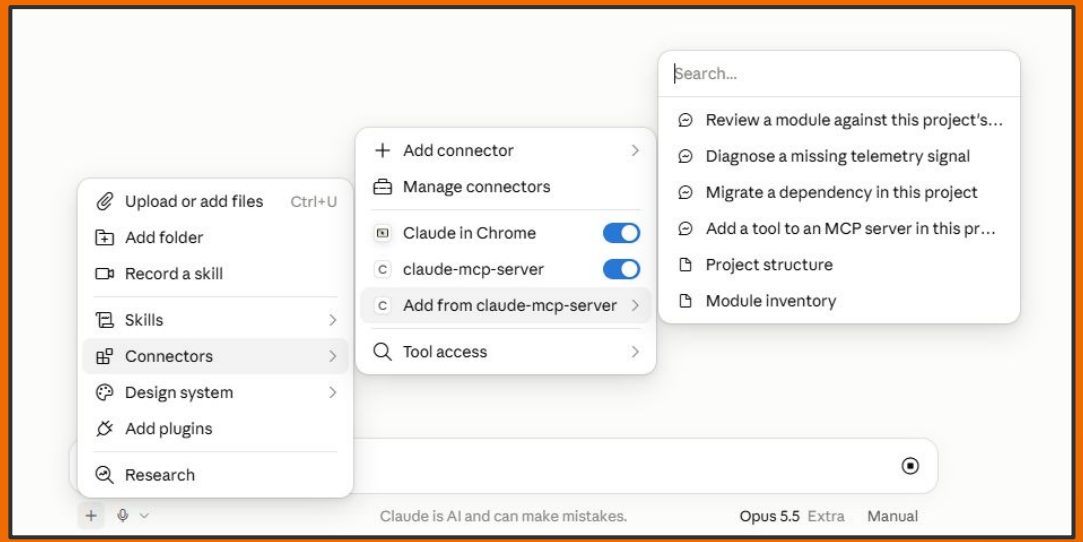
- Four only read, four write, one deletes.
- Every tool is access you hand to a model. Give only what the job needs.
- Each tool should work only inside the area it was built for, and nowhere else.
- Here that area is one project folder, allowed file types only, and never .git or target.

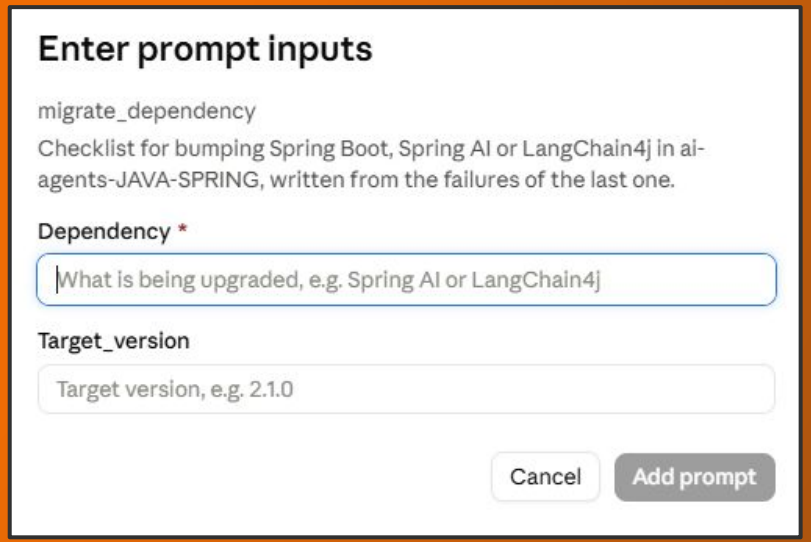
TOOL	WHAT IT DOES
READ	
`read_file`	read one file
`list_files`	list a directory
`get_project_structure`	show the directory tree
`search_in_files`	find text in the project
WRITE	
`write_file`	overwrite an existing file
`create_file`	create a new file
`move_file`	move or rename a file
`move_directory`	move or rename a directory
DELETE	
`delete_file`	delete a file

More than tools

Tools are one of three primitives, plus completions

Here are the other two, as Claude Desktop shows them.





Resources

A resource can be a view

`project://modules` is not a file. The server builds it on every read, from each module's own application.yml.

```
@McpResource(
        uri = "project://modules",
        name = "Module inventory",
        title = "Modules, ports and application names",
        description = "Every Maven module with the port it listens on and its "
                + "spring.application.name — read from the modules' own "
                + "application.yml files, not from a hand-maintained table.",
        mimeType = "text/plain")
public String modules() {                                   // computed on every read
    return telemetry.traced("resource", "project://modules", this::buildModuleInventory);
}
```

Prompts

A prompt is a message template the person fills in.

The person picks it from a menu.

`@McpArg` becomes the form field, and the server returns the text the model will follow.

```
@McpPrompt(
        name = "review_module",
        title = "Review a module against this project's conventions",
        description = "Structured review of one module: observability wiring, tool "
                + "contracts, configuration and comments.")
public String reviewModule(
        @McpArg(name = "module",
                description = "Module to review, e.g. spring-ai-agent-mcp",
                required = true) String module) {           // becomes a form field

    return telemetry.traced("prompt", "review_module", () -> """
            Review the module `%s` against the conventions of ai-agents-JAVA-SPRING.
            …
            Report findings ordered by risk, and separate "I checked this" from
            "I suspect this".
            """.formatted(module, module));
}
```

Completions

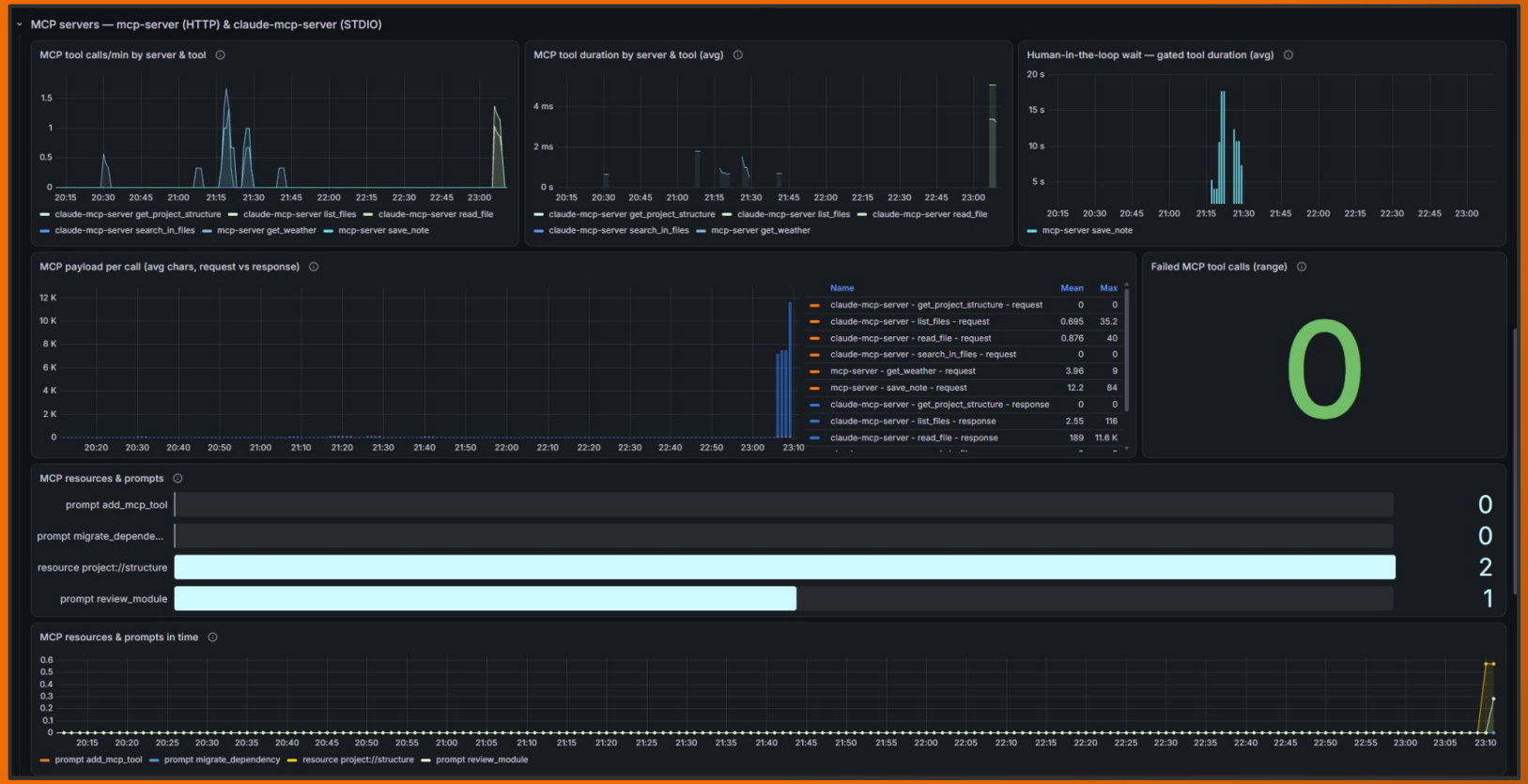
Completions help the person typing

The server suggests module names while the field is being filled in.

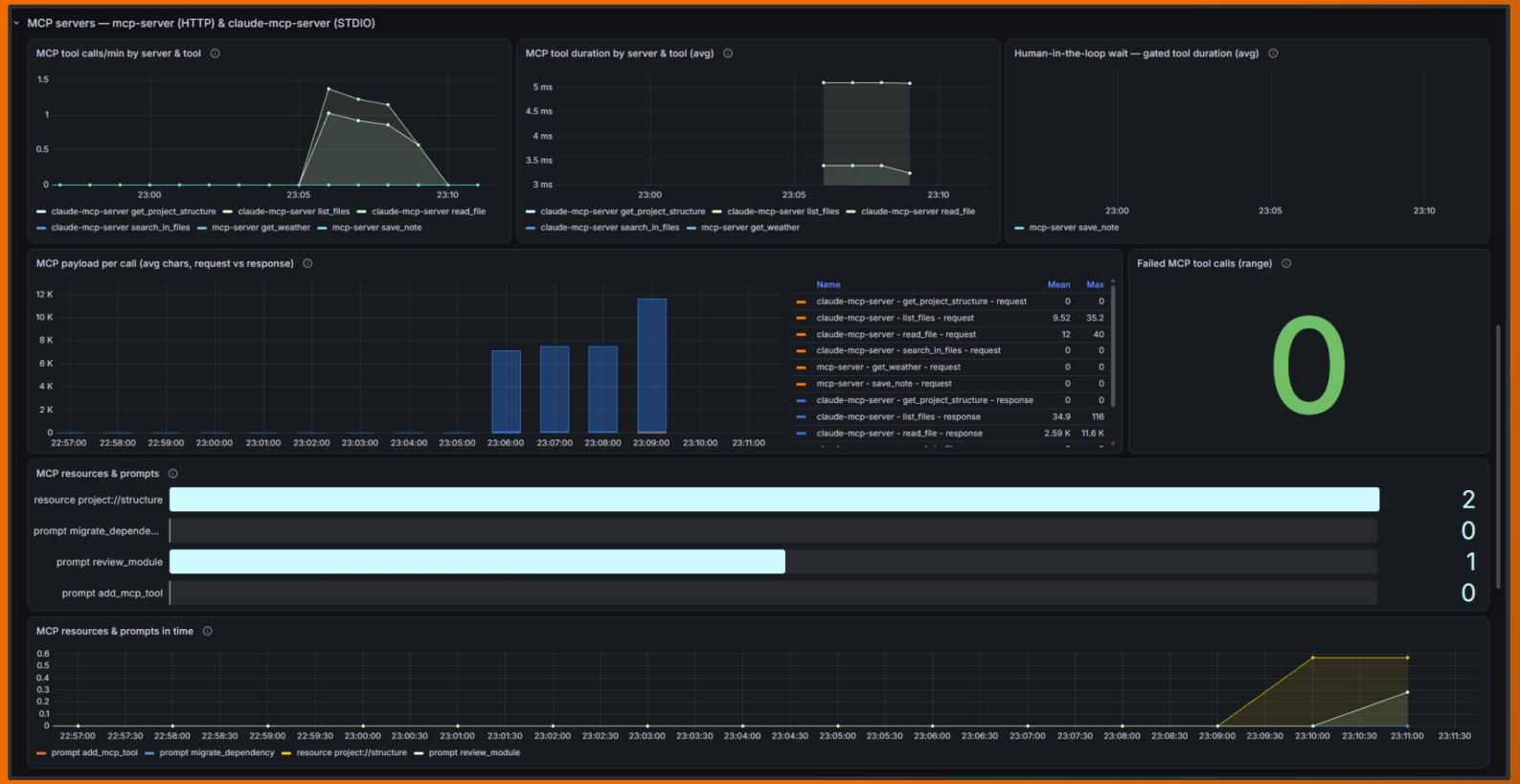
```
/** Completes the single argument of the review_module prompt. */
@McpComplete(prompt = "review_module")
public List<String> completeReviewModule(String prefix) {
    return filter(moduleNames(), prefix);                    // modules discovered, not listed
}

private static List<String> filter(List<String> candidates, String prefix) {
    String needle = prefix == null ? "" : prefix.toLowerCase();
    return candidates.stream()
            .filter(candidate -> candidate.toLowerCase().startsWith(needle))
            .limit(MAX_SUGGESTIONS)
            .toList();
}
```

Last 3 hours


MCP servers — mcp-server (HTTP) & claude-mcp-server (STDIO)
MCP tool calls/min by server & tool
claude-mcp-server get_project_structure
claude-mcp-server list_files
claude-mcp-server read_file
claude-mcp-server search_in_files
mcp-server get_weather
mcp-server save_note
MCP tool duration by server & tool (avg)
Human-in-the-loop wait — gated tool duration (avg)
MCP payload per call (avg chars, request vs response)
Name
Mean
Max
claude-mcp-server - get_project_structure - request 0 0
claude-mcp-server - list_files - request 0.695 35.2
claude-mcp-server - read_file - request 0.876 40
claude-mcp-server - search_in_files - request 0 0
mcp-server - get_weather - request 3.96 9
mcp-server - save_note - request 12.2 84
claude-mcp-server - get_project_structure - response 0 0
claude-mcp-server - list_files - response 2.55 116
claude-mcp-server - read_file - response 189 11.6 K
Failed MCP tool calls (range)
0
MCP resources & prompts
prompt add_mcp_tool
prompt migrate_depende...
resource project://structure
prompt review_module
0
0
2
1
MCP resources & prompts in time
prompt add_mcp_tool
prompt migrate_dependency
resource project://structure
prompt review_module

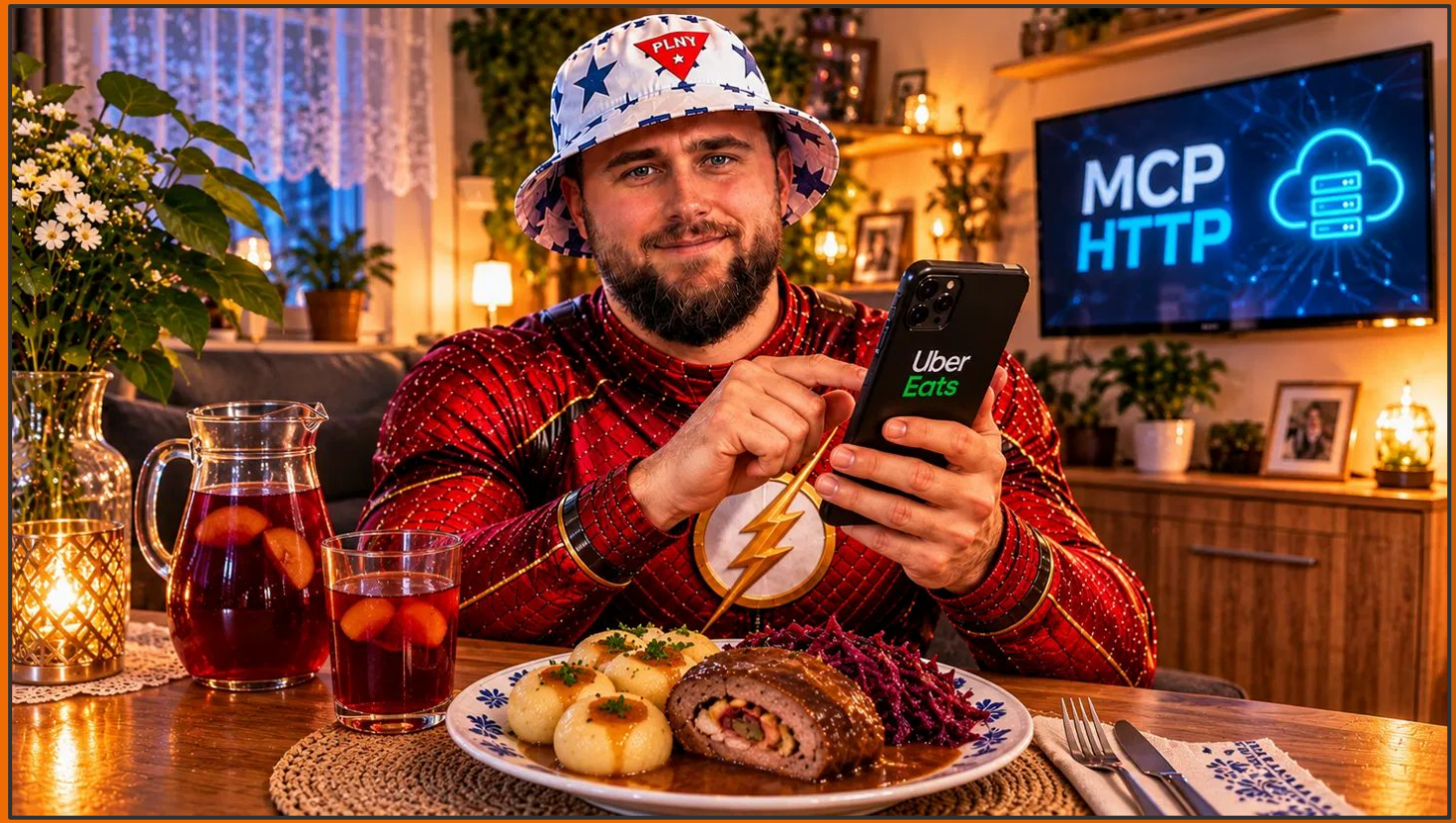
Last 15 minutes



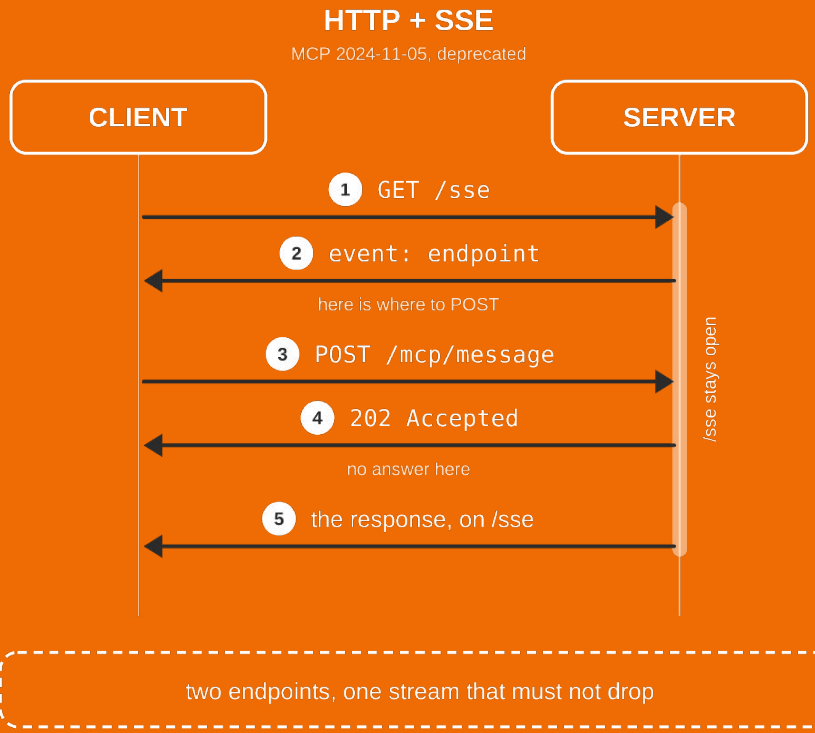
From a Spring bean to Claude Desktop

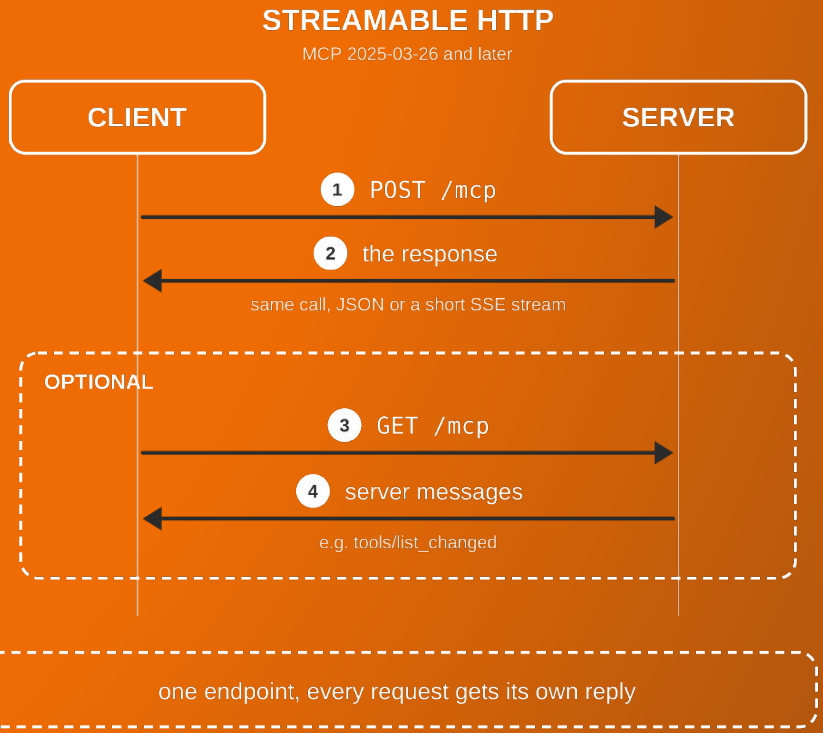
1. Add spring-ai-starter-mcp-server to **pom.xml**
2. Put @McpTool on a method of a Spring bean
3. Turn STDIO on → spring.ai.mcp.server.stdio: true
4. Keep stdout silent → no banner, no web server, logs to a file
5. Build the jar → add java -jar to claude_desktop_config.json and restart Claude Desktop

MCP Streamable HTTP



Why SSE was replaced





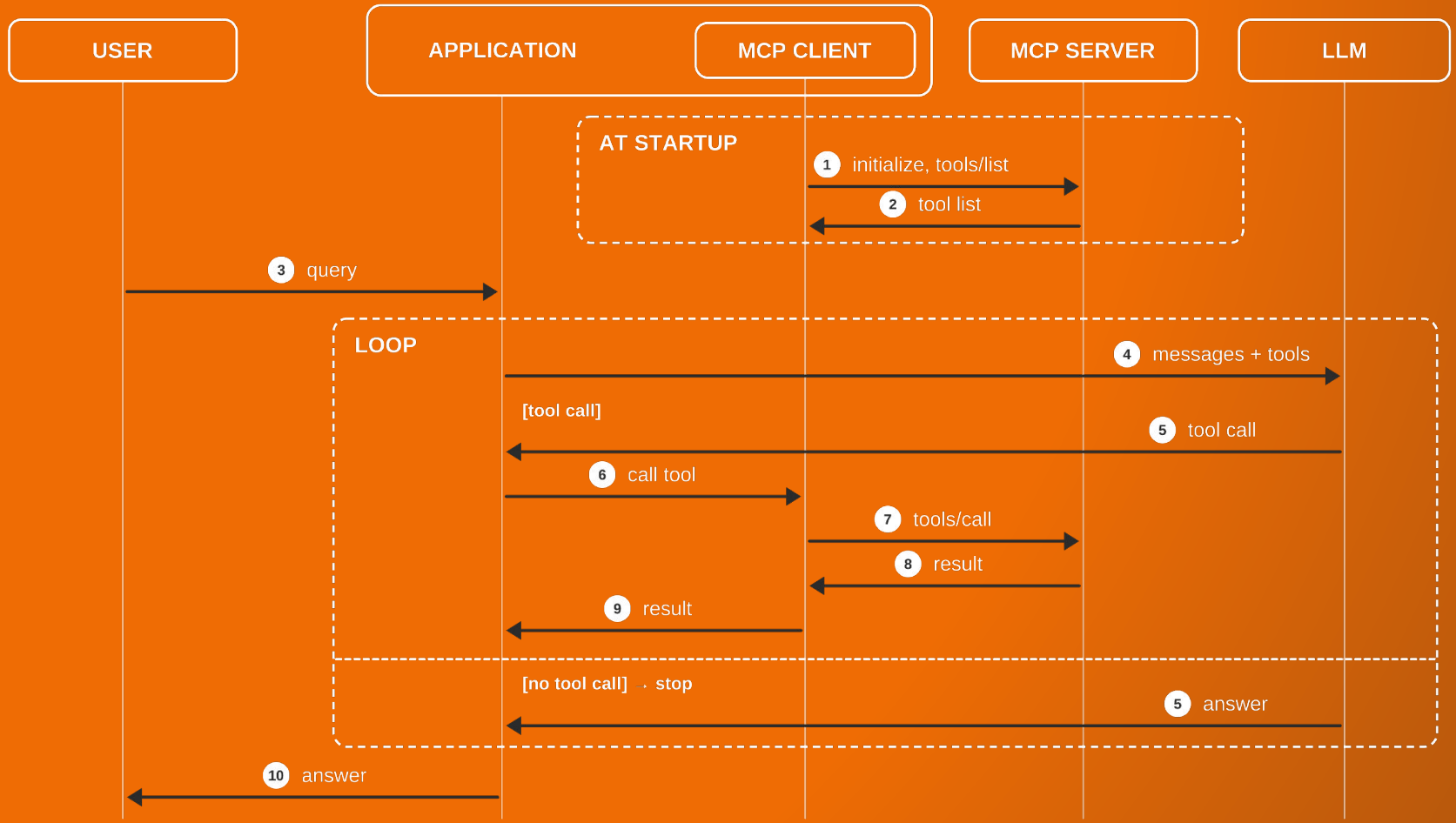
MCP over HTTP

A normal Spring Boot web app.

Same @McpTool annotation, a different transport.

```yaml
server:
  port: 8081                          # a normal web app, clients need a URL

spring:
  ai:
    mcp:
      server:
        name: ai-sandbox-mcp-server
        protocol: STREAMABLE          # the default in Spring AI 2.0, SSE is deprecated
        streamable-http:
          mcp-endpoint: /mcp          # one endpoint, both directions
        capabilities:                 # advertise only what exists
          tool: true
          resource: false
          prompt: false
          completion: false
```


USER
APPLICATION
MCP CLIENT
MCP SERVER
LLM
AT STARTUP
1 initialize, tools/list
2 tool list
3 query
LOOP
4 messages + tools
[tool call]
5 tool call
6 call tool
7 tools/call
8 result
9 result
[no tool call] → stop
5 answer
10 answer

STDIO vs Streamable HTTP

	STDIO `claude-mcp-server`	**STREAMABLE HTTP** `mcp-server`
STARTER	`spring-ai-starter-mcp-server`	`spring-ai-starter-mcp-server-webmvc`
TRANSPORT	`stdio: true`	`protocol: STREAMABLE`
CLIENT CONFIG	`"command": "java"` the client runs the jar	`http://localhost:8081/mcp` the client only needs a URL
LIFECYCLE	the client starts it and kills it	runs on its own, start it first
WHO CAN REACH IT	only the process that started it	anyone who can reach the port, here only localhost

The tool every demo uses

A tool runs where the server runs.

Over HTTP, `get_weather` runs inside mcp-server, not inside your agent.

```
@McpTool(name = "get_weather",                          // same annotation as in claude-mcp-server
        description = "Get current weather for a city. "
        + "Returns temperature, conditions and humidity.",
        annotations = @McpTool.McpAnnotations(readOnlyHint = true))
public String get_weather(
        @McpToolParam(description = "City name", required = true)
        String city) {
    return telemetry.traced("get_weather", "city=" + city, len(city), () -> {
        log.info("[MCP] get_weather city={}", city);
        return String.format("Weather in %s: 22C, partly cloudy, humidity 65%%", city);
    });
}
```

MCP Client → Spring AI

One server = one client

Every entry under connections becomes its own MCP client.

Each one connects at startup, and all their tools end up in one ToolCallbackProvider.

```yaml
spring:
  ai:
    mcp:
      client:
        enabled: true
        type: SYNC
        streamable-http:          # must match the server's transport
          connections:
            java-mcp-server:
              url: http://localhost:8081
              endpoint: /mcp
```

Handing the tools to the model

```
private final ChatClient chatClient;
private final ToolCallbackProvider mcpToolCallbackProvider;   // autoconfigured from the yml

public String orchestrate(String message) {
    log.info("Spring AI Orchestrator: {}", message);

    return chatClient.prompt()
            .system("""
                    You are an orchestrator agent.
                    …
                    """)
            .user(message)
            .advisors(new SimpleLoggerAdvisor())
            .tools(mcpToolCallbackProvider)       // takes any kind of tool, even mixed
            .call()
            .content();
}
```

Spring AI needs two hooks

Spring Boot does not trace the JDK HttpClient inside the MCP client.

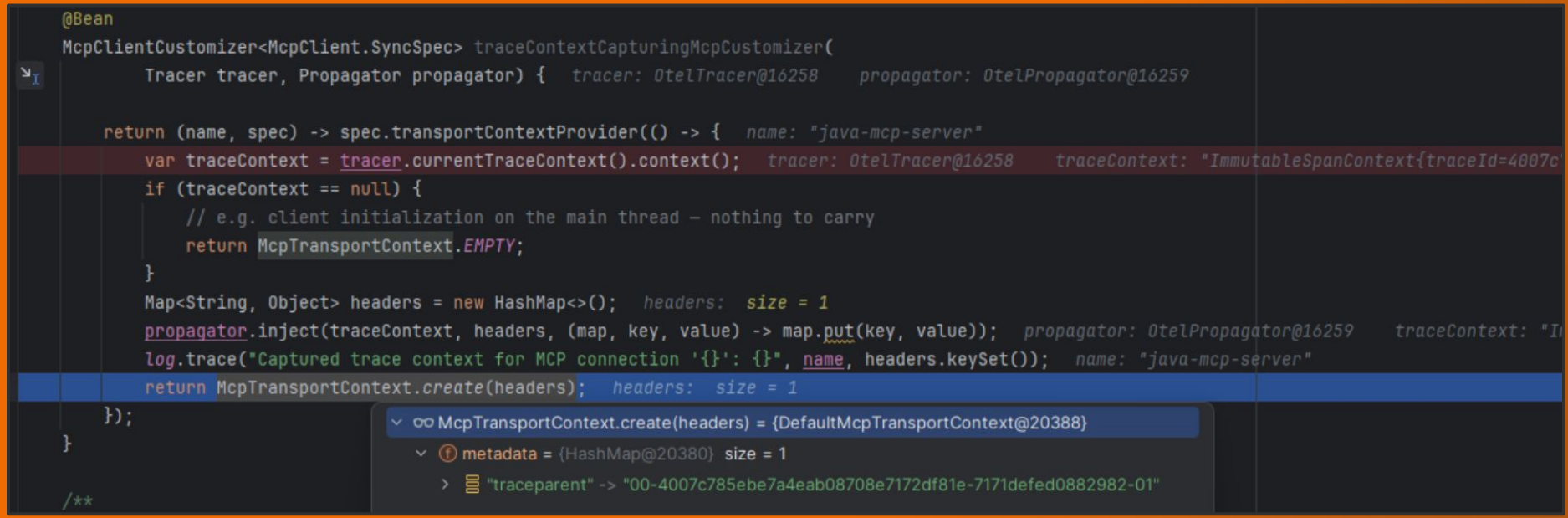
Step 1 runs on the calling thread and copies the span into McpTransportContext.

```
@Bean
McpClientCustomizer<McpClient.SyncSpec> traceContextCapturingMcpCustomizer(
        Tracer tracer, Propagator propagator) {

    return (name, spec) -> spec.transportContextProvider(() -> {
        var traceContext = tracer.currentTraceContext().context();
        if (traceContext == null) {
            // e.g. client initialization on the main thread — nothing to carry
            return McpTransportContext.EMPTY;
        }
        Map<String, Object> headers = new HashMap<>();
        propagator.inject(traceContext, headers, (map, key, value) -> map.put(key, value));
        log.trace("Captured trace context for MCP connection '{}': {}", name, headers.keySet());
        return McpTransportContext.create(headers);
    });
}
```

The copy holds the traceparent

The copy holds one header, the traceparent of the current span.



Step 2 writes the header

It runs right before every POST to /mcp.

It reads the span from the ThreadLocal, and uses the copy when the call resumes on another thread.

```
@Bean
McpClientCustomizer<HttpClientStreamableHttpTransport.Builder> traceContextPropagatingMcpCustomizer(
        Tracer tracer, Propagator propagator) {

    return (name, transportBuilder) ->
            transportBuilder.httpRequestCustomizer(requestCustomizer(tracer, propagator));
}

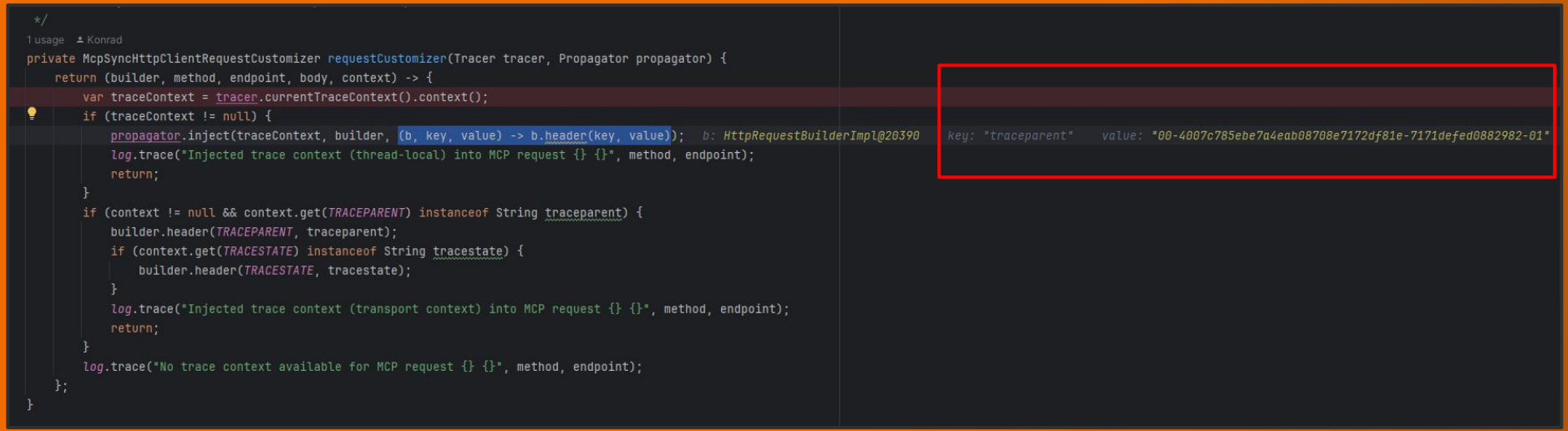
private McpSyncHttpClientRequestCustomizer requestCustomizer(Tracer tracer, Propagator propagator) {
    return (builder, method, endpoint, body, context) -> {
        var traceContext = tracer.currentTraceContext().context();
        if (traceContext != null) {
            propagator.inject(traceContext, builder, (b, key, value) -> b.header(key, value));
            log.trace("Injected trace context (thread-local) into MCP request {} {}", method, endpoint);
            return;
        }
        if (context != null && context.get(TRACEPARENT) instanceof String traceparent) {
            builder.header(TRACEPARENT, traceparent);
            if (context.get(TRACESTATE) instanceof String tracestate) {
                builder.header(TRACESTATE, tracestate);
            }
            log.trace("Injected trace context (transport context) into MCP request {} {}", method, endpoint);
            return;
        }
        log.trace("No trace context available for MCP request {} {}", method, endpoint);
    };
}
```

COPY

Same thread, same traceparent

The ThreadLocal branch ran, so the copy was not needed.

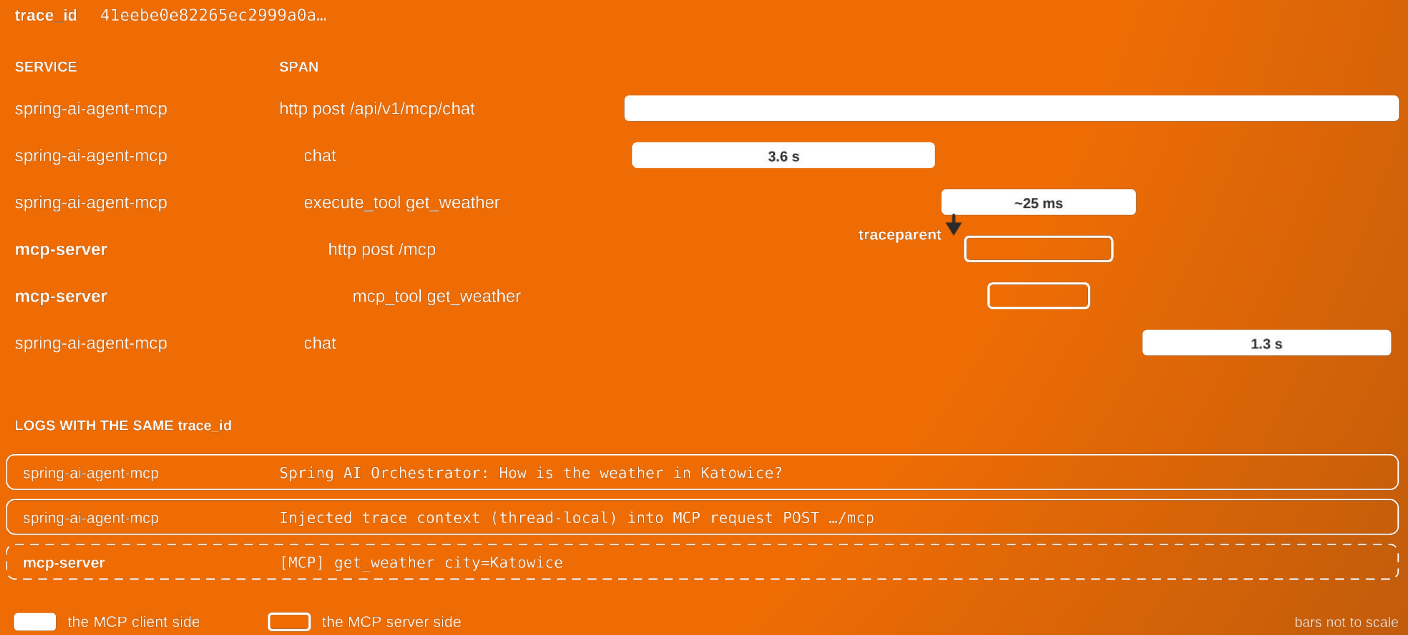
Same span, same traceparent as in step 1



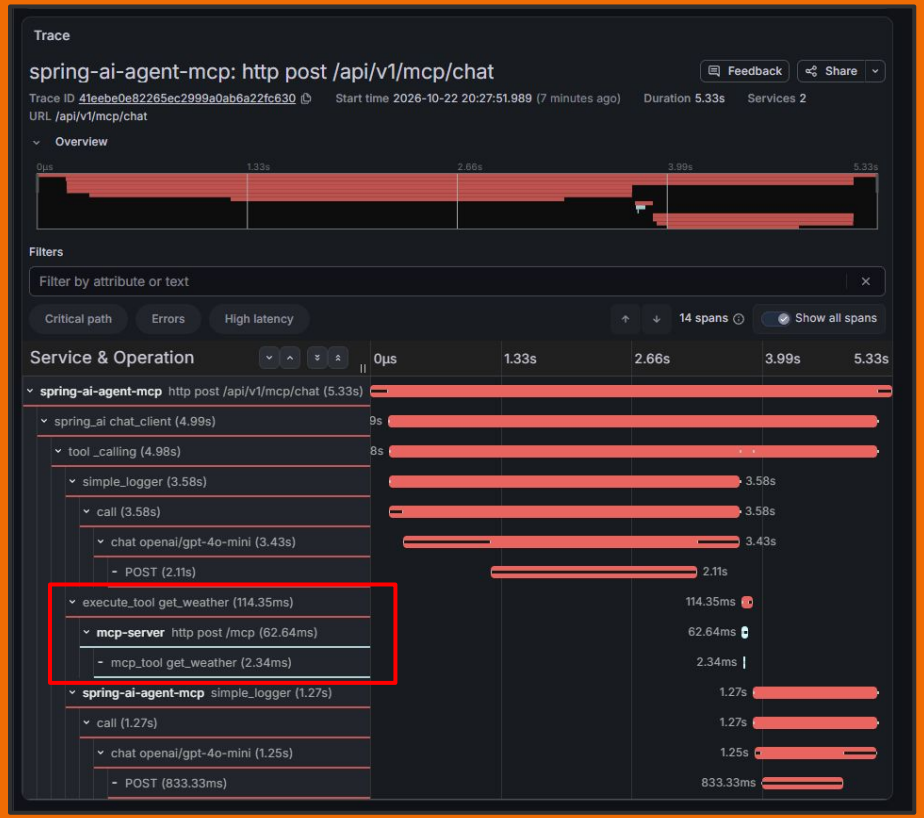
One trace, client and server

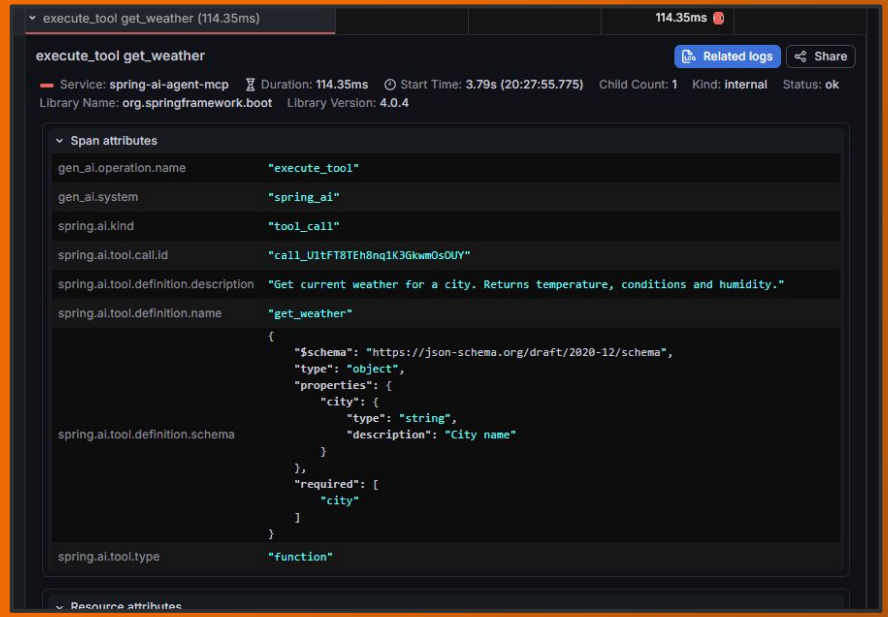
The customizer in this repo adds the traceparent header to every POST to /mcp.

MCP server continues the same trace, so spans and logs from both processes share one trace_id.



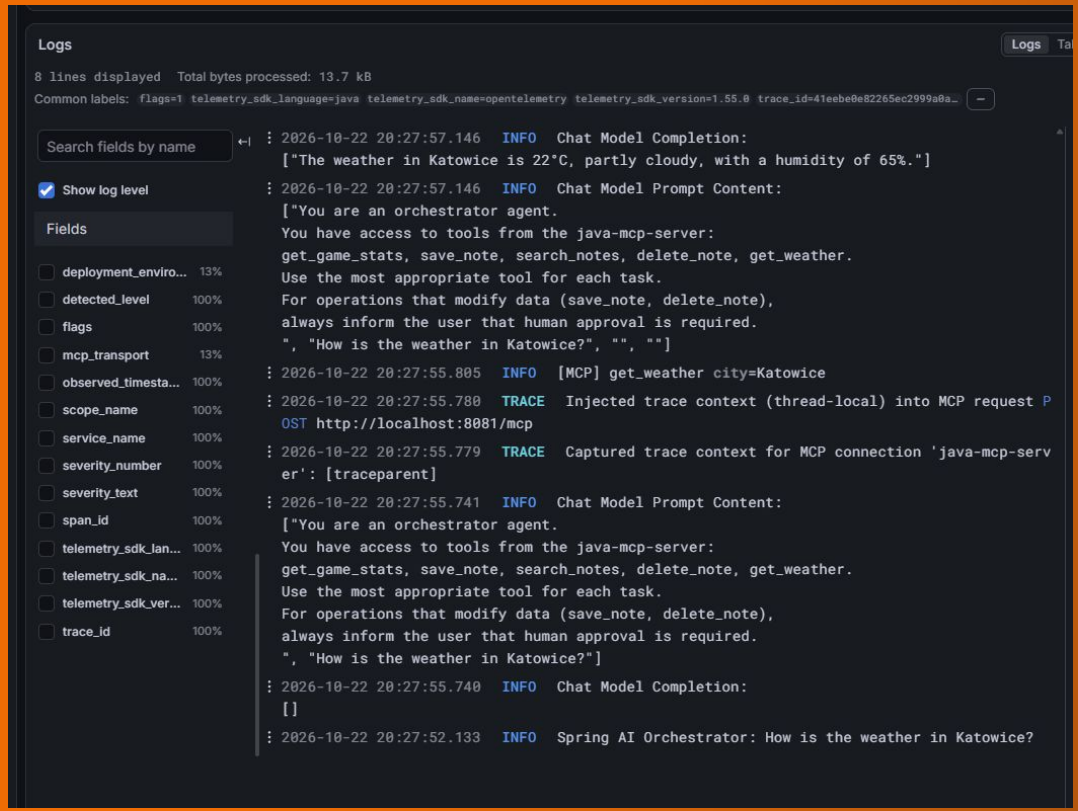
One trace, two services





Same trace_id in the logs

Agent and mcp-server lines in one query.



The same call in metrics



MCP Client → LangChain4j

One server, one client here too.

LangChain4j ships its own MCP client, not the official Java SDK.

It is built in code, and McpToolProvider puts the tools of all clients into one list.

LangChain4j builds it in code

```
public OrchestratorService(
        ChatModel model,
        @Qualifier("javaMcpClient") McpClient javaMcpClient,        // whichever variant was built
        ObjectProvider<Tracer> tracerProvider) {

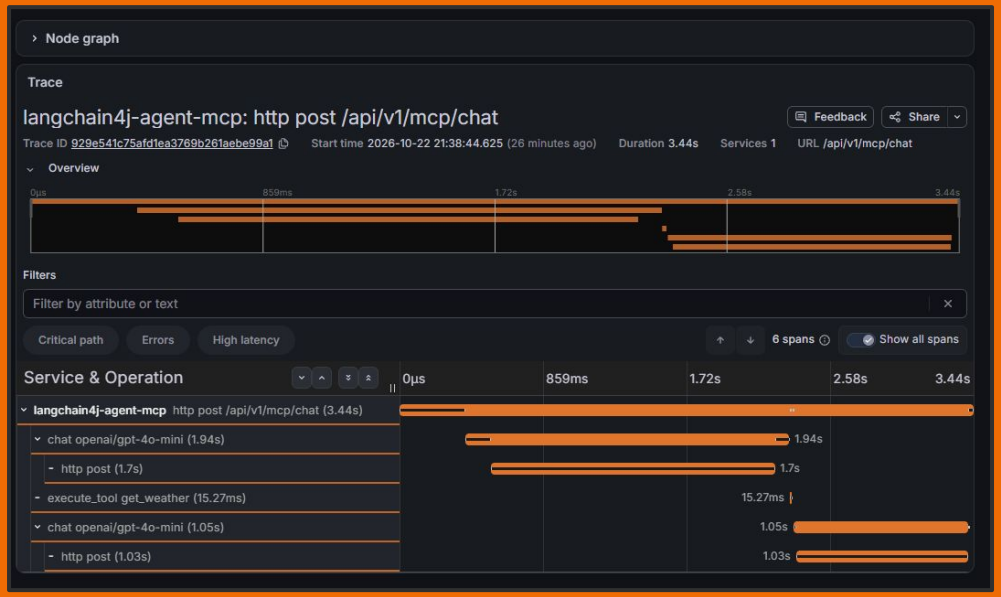
    ToolProvider toolProvider = new TracingToolProvider(
            McpToolProvider.builder().mcpClients(List.of(javaMcpClient)).build(),
            tracerProvider.getIfAvailable());

    this.assistant = AiServices.builder(OrchestratorAssistant.class)
            .chatModel(model)
            .toolProvider(toolProvider)
            .chatMemoryProvider(memoryId -> MessageWindowChatMemory.withMaxMessages(MEMORY_WINDOW))
            .build();
}
```

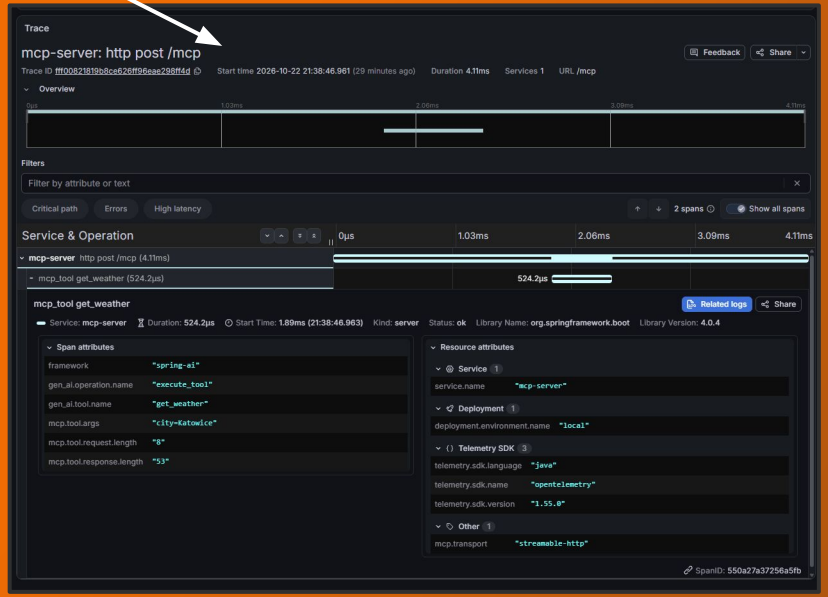
What we miss?

```
{ resource.service.name = "mcp-server" && name = "mcp_tool get_weather" }
```

Start time 2026-10-22 21:38:44.625



Start time 2026-10-22 21:38:46.961



Why two traces?



WHAT LANGCHAIN4J SENDS

```
POST /mcp
Content-Type: application/json
Accept: application/json,text/event-stream
Mcp-Session-Id: …
```

WHAT WOULD JOIN THE TRACE

```
traceparent: 00-<trace-id>-<span-id>-01
```

one header, written from the current span

Spring Boot traces RestClient and WebClient.
The MCP clients use the JDK HttpClient, so nobody writes the header unless your code does.

Two beans, one name

```java
@Bean(name = "javaMcpClient")
@ConditionalOnProperty(name = "lc4j.mcp.trace-propagation.enabled", havingValue = "false",
        matchIfMissing = true)
public McpClient javaMcpClient() {
    log.info("MCP trace propagation disabled — a tool call produces two traces");
    return client(transport().build());
}

@Bean(name = "javaMcpClient")
@ConditionalOnProperty(name = "lc4j.mcp.trace-propagation.enabled", havingValue = "true")
public McpClient javaMcpClientWithTracePropagation(Tracer tracer, Propagator propagator) {
    log.info("MCP trace propagation ENABLED — traceparent is sent on every POST /mcp");
    return client(transport()
            .customHeaders(() -> traceHeaders(tracer, propagator))
            .build());
}
```

Transport and client

```
private static StreamableHttpMcpTransport.Builder transport() {
    return new StreamableHttpMcpTransport.Builder()
            .url("http://localhost:8081/mcp")
            .logRequests(true)
            .logResponses(true);
}

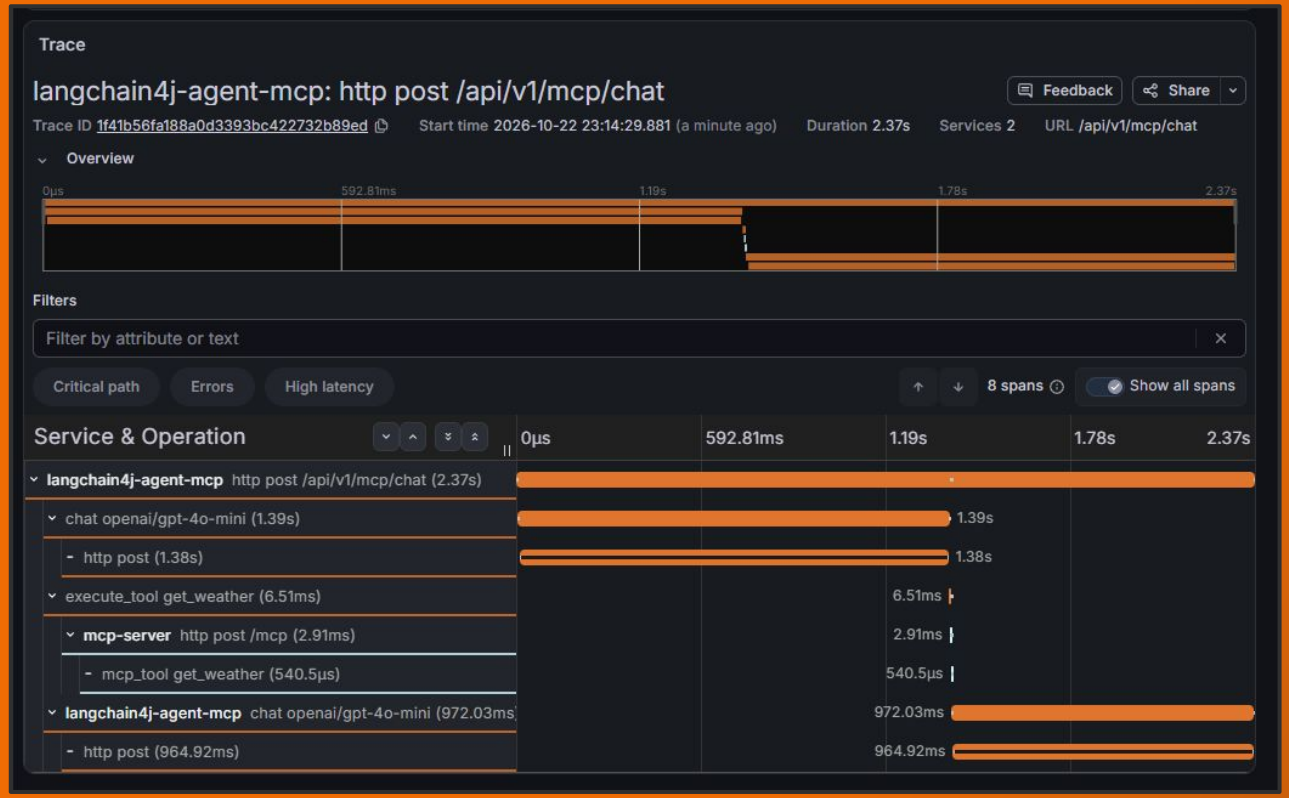
private static McpClient client(McpTransport transport) {
    return new DefaultMcpClient.Builder()
            .transport(transport)
            .clientName("langchain4j-java-client")
            .clientVersion("1.0")
            .build();
}
```

One header supplier

Called for every request, on the thread that runs the tool.

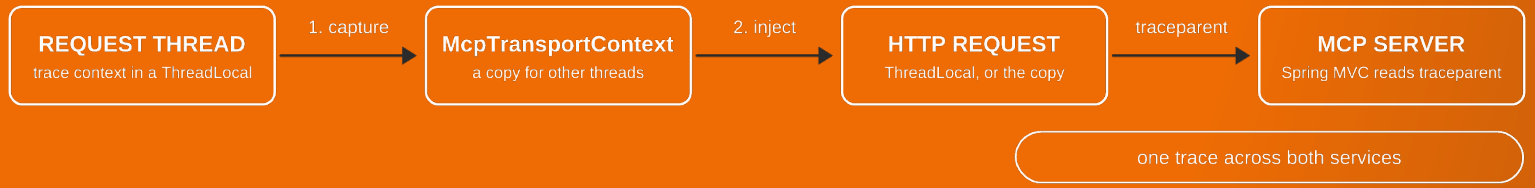
```
private static Map<String, String> traceHeaders(Tracer tracer, Propagator propagator) {
    TraceContext context = tracer.currentTraceContext().context();
    if (context == null) {
        return Map.of();                // initialize and health-check pings, no span
    }
    Map<String, String> headers = new HashMap<>();
    propagator.inject(context, headers, (map, key, value) -> map.put(key, value));
    return headers;
}
```

After the switch



Where the trace context gets lost

Spring AI



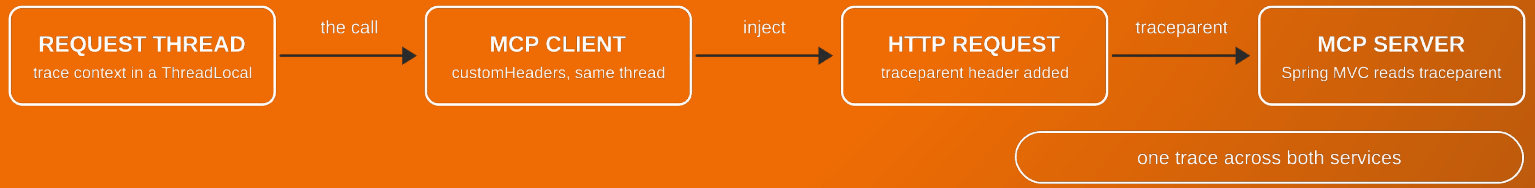
LangChain4j → switch off, the default

`lc4j.mcp.trace-propagation.enabled=false`



LangChain4j → switch on

`lc4j.mcp.trace-propagation.enabled=true`



Two clients, one protocol

	SPRING AI `spring-ai-agent-mcp`	LANGCHAIN4J `langchain4j-agent-mcp`
CONFIGURED IN	application.yml	Java code
MCP CLIENT	the official MCP Java SDK	its own, langchain4j-mcp
ONE PER SERVER	an entry under connections	a DefaultMcpClient bean
TOOLS TO THE MODEL	`.tools(mcpToolCallbackProvider)`	`.toolProvider(toolProvider)`
AT STARTUP	connects, or the app fails	connects, or the app fails
TOOL TIMEOUT	20 s `spring.ai.mcp.client.request-timeout`	60 s `toolExecutionTimeout`
SAME TOOL NAME TWICE	renamed to alt_1_<name> a WARN in the log, the model sees the new name	`IllegalConfigurationException` on the call, not at startup
TRACE CONTEXT	always on in this repo `McpTracePropagationConfig`	a switch, off by default `lc4j.mcp.trace-propagation.enabled`

What 2026-07-28 changes

	2025-11-25 what this repo speaks	**2026-07-28** the current revision
HANDSHAKE	initialize, then initialized	gone version and capabilities travel in every request's _meta
SESSION	the Mcp-Session-Id header	gone, no sessions
SERVER NOTIFICATIONS	the GET stream	subscriptions/listen
SERVER ASKS THE CLIENT	a request from the server elicitation	multi round-trip requests (MRTR) the server answers input_required, the client retries
ROOTS, SAMPLING, LOGGING	part of the spec	deprecated

Where it breaks?

WHAT WENT WRONG	WHAT YOU SAW
`javac without -parameters` @McpToolParam has no name attribute	the schema says arg0, every argument arrives as null
`logging.file.name: logs/…` Claude Desktop starts the JVM in C:\WINDOWS\system32	the log file is not where you look
`resource: true` advertised in the handshake, nothing registered	resources/list returns an empty array
`sse: vs streamable-http:` client and server on different transports	20 s hang, then "Did not observe any item"
`agent started before mcp-server` the MCP client initialises eagerly at startup	ConnectException, the agent never starts
`@Bean …RequestCustomizer` Spring AI 2.0 no longer reads a standalone bean	two traces, every piece looks fine
`allow-bean-definition-overriding` our mcpToolCallbacks bean replaced	zero tools, input_tokens gave it away

See you in part Ⅳ next week!

